

Sisteme de Operare

Cap 10. Alocarea memoriei. Memoria virtuală (II)

May 10, 2023

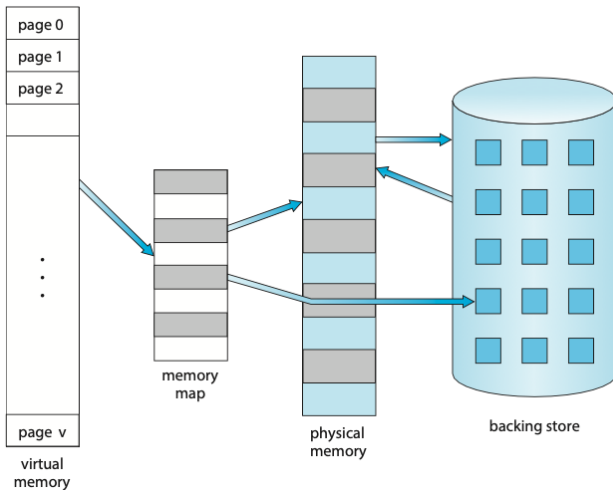
- 1 Fundamente
- 2 Paginarea la cerere
- 3 Înlocuirea paginilor
- 4 Trashing
- 5 Alocarea memoriei de către kernel

- 1 Fundamente
- 2 Paginarea la cerere
- 3 Înlocuirea paginilor
- 4 Trashing
- 5 Alocarea memoriei de către kernel

Codul programului nu este folosit uniform

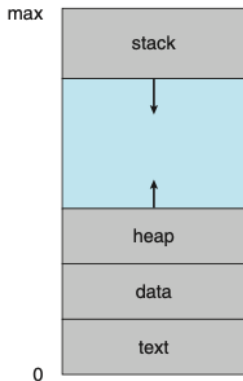
- multiprogramarea presupune păstrarea mai multor procese în memorie
- folosirea memoriei virtuale elimină necesitatea păstrării întregului proces în memorie, pentru a putea fi executat
- memoria virtuală separă spațiul logic de adrese al programului de spațiul fizic de adrese
- poate descrește performanța dacă este folosită neglijent
- programele prezintă:
 - programele au cod pentru tratarea condițiilor de eroare, cod care este apelat rareori
 - listelor, array-urilor, li se alocă mai multă memorie decât necesarul
 - multe părți din program sunt arareori folosite
- execuția unui program parțial încărcat în memorie permite:
 - lungimi oricât de mari pentru programe
 - încărcarea și mai multor programe simultan
 - mai puține operații I/O pentru încărcarea sau swapping-ul unor pagini de memorie

Memoria virtuală



- memoria virtuală poate fi mult mai mare decât memoria fizică

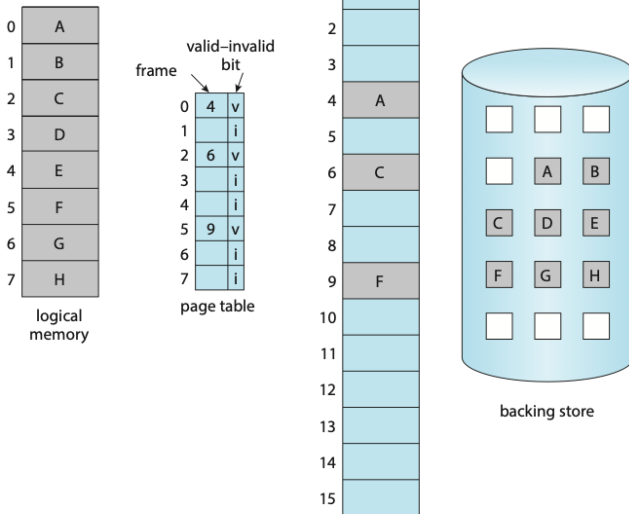
Spațiu de adrese "găurit" (sparse)



- spațiul de adrese dintre stivă și heap va necesita mapare în pagini fizice doar pe măsură ce este folosit
- spațiul adrese virtual permite partajarea bibliotecilor încărcate dinamic precum biblioteca C standard
- procesele pot partaja aceeași zonă fizică de memorie deși fiecare "vede" acea zonă cu propria adresă logică (diferită de cea a celui alt)
- paginile pot fi partajate rapid la crearea unui proces cu apelul de sistem **fork()**

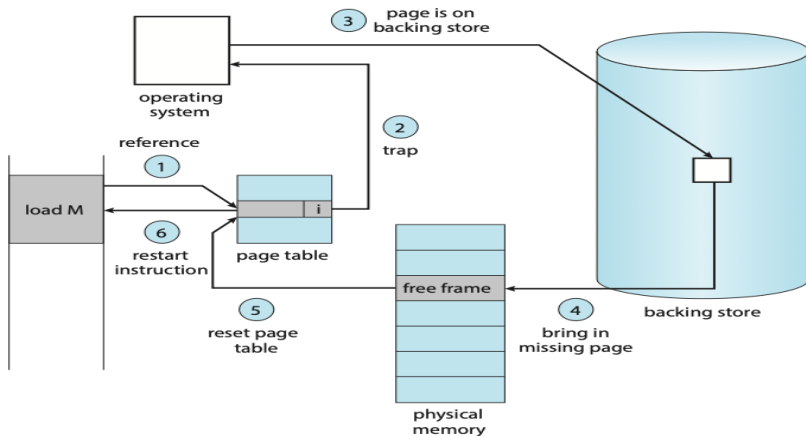
- 1 Fundamente
- 2 **Paginarea la cerere**
- 3 Înlocuirea paginilor
- 4 Trashing
- 5 Alocarea memoriei de către kernel

Păstrarea parțială a paginilor în memorie



Funcționarea memoriei virtuale

- fiecare intrare din tabela de pagini conține un bit valid/invalid ce indică dacă pagina este mapată într-un frame fizic sau nu
- accesul la o pagină invalidă va genera un **page fault**, adică o întrerupere HW urmată de un trap în SO



Funcționarea memoriei virtuale (2)

- **pure demand paging**: situația ipotetică în care un proces pornește fără a avea pagini alocate și le încarcă pe măsură ce are nevoie de ele
- **localizarea referințelor**: programele au tendința să acceseze anumite zone de memorie în decursul rulării, numărul de page fault-uri generate scad în timp
- după un page fault, instrucțiunea care l-a generat este re-executată, procesul este repornit exact în același loc și același context

Alocarea frame-urilor

- frame-urile disponibile sunt menținute într-o listă înlănțuită denumită **free frame list**
- SO alocă frame-urile folosind **zero-fill on demand**, adică frame-urile sunt completate cu valoarea zero înainte de a fi alocate, din rațiuni de securitate
- lista cu frame-uri disponibile este plină doar la boot-area sistemului, după care scade; la golire, sau mai degrabă la atingerea unui nivel minim, se aplică diverse tehnici de "repopulare" prin recuperarea frame-urilor utilizate

Performanța sistemului de paginare

- paginarea la cerere poate afecta performanța sistemului
- **timpul efectiv de acces** la memorie este mai mare decât timpul real de acces la memoria fizică

$$\text{timp efectiv de acces} = (1 - p) \times t_{mem} + p \times t_{\text{page fault}}$$

- t_{mem} este mic, tipic 10 [ns]; probabilitatea unui page fault, $0 \leq p \leq 1$, este aproape de zero
- un page fault determină următoarele acțiuni: 1. trap în SO; 2. salvarea regiștrilor și a stării procesului; 3. stabilirea că întreruperea este un page fault; 4. verificarea validității referinței (excluderea pointer-ului invalid) și căutarea paginii în spațiul secundar de stocare; 5. așteptarea servirii cererii de aducere a paginii în memorie; 6. alocarea CPU-lui altui proces în timpul așteptării; 7. sosirea întreruperii de la I/O, cum că transferul s-a realizat; 8. actualizarea tabelii de pagini cum că pagina este acum în memorie; 9. așteptarea ca procesul să fie din nou alocat CPU-ului spre execuție.

Rata de page fault-uri

- presupunem un timp de servire a page fault-ului de 8 [ms] și un timp de acces la memorie de 200 [ns]

$$\begin{aligned}
 \text{timp efectiv de acces} &= (1 - p) \times 200 + p \times 8[ms] \\
 &= (1 - p) \times 200 + p \times 8.000.000 \\
 &= 200 + 7.999.800 \times p
 \end{aligned}$$

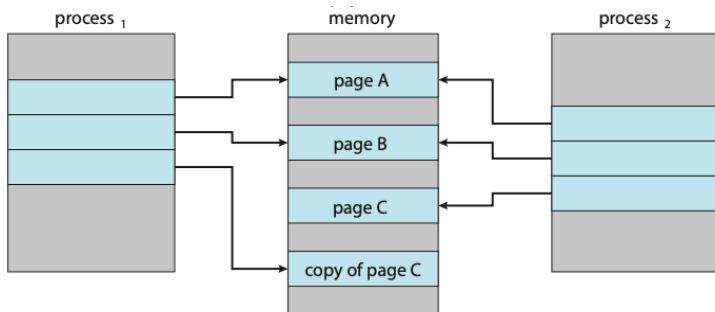
- dacă se produce o rată de page fault-uri de 1 page fault la 1000 de accese la memorie, timpul efectiv de acces este de 8.2 [μs], calculatorul este încetinit de 40 de ori
- dacă dorim o degradare a performanței de mai puțin de 10%:

$$\begin{aligned}
 220 &> 200 + 7.999.800 \times p \\
 20 &> 7.999.800 \times p \\
 p &< 0.0000025
 \end{aligned}$$

- mai puțin de 1 page fault la 400.000 accese

Copy-on-write

- pentru a minimiza numărul de frame-uri folosite, la crearea unui proces cu `fork()` nu se copiază spațiul de adrese al părintelui, ci se marchează paginile ca partajate; dacă unul din procese va scrie într-o pagină partajată, pagina va fi copiată iar modificarea sa se va realiza într-o pagină duplicat proprie procesului
- apelul sistem `vfork()` nu face acest copy-on-write, poate suprascrie pagini, însă este foarte rapid



- 1 Fundamente
- 2 Paginarea la cerere
- 3 Înlocuirea paginilor**
- 4 Trashing
- 5 Alocarea memoriei de către kernel

Tehnici de bază

- dacă nu există frame-uri libere, se găsește unul nefolosit
- conținutul său se scrie în memoria secundară (SSD), apoi se marchează tabelele de pagini care în conțineau că pagina respectivă lipsește; frame-ul este refolosit
- procesul de înlocuire determină două accese, unul de scriere și unul de citire; numărul de accese se reduce prin folosirea unui **dirty bit**, ce marchează dacă pagina a fost sau nu scrisă
- o pagină nescrisă (ex. cod) poate fi aruncată fără a fi salvată, conținutul ei există deja în filesystem
- procesele au nevoie de un **algorithm de alocare a paginilor** (câte pagini alocăm unui proces) și de un **algorithm de înlocuire a paginilor** (ce frame-uri se refolosesc dintre cele deja ocupate)
- evaluarea algoritmilor de înlocuire a paginilor se face prin rularea lor pe un așa-numit **reference string** de locații de memorie accesate de proces, urmat de numărarea page fault-urilor
- nu are sens să folosim adresele referite, ci doar numărul paginii - toate adresele cu un anumit prefix se vor afla într-o singură pagină

Algoritmul FIFO de înlocuire a paginilor

- asociază fiecărei pagini timpul la care ea a fost încărcată în memorie
- pagina cu cel mai vechi timp este aleasă pentru înlocuire
- este suficient să menținem o structură FIFO: când aducem o pagină o vom insera la coadă (tail), iar pagina ce se înlocuiește va fi mereu cea din capăt (head)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2																
	0	0	0																
		1	1																

page frames

Algoritmul FIFO - raționament alternativ

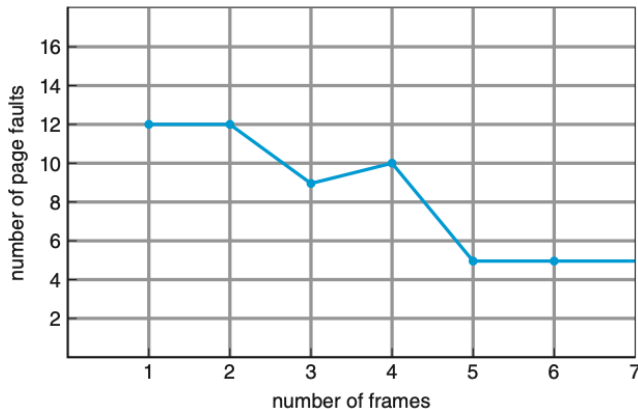
- mutăm în FIFO paginile pe măsură ce se adaugă unele noi

p	p	p	p	-	p	p	p	p	p	p	-	-	p	p	-	-	p	p
7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0
7	0	1	2	2	3	0	4	2	3	0	0	0	1	2	2	2	7	0
-	7	0	1	1	2	3	0	4	2	3	3	3	0	1	1	1	2	7
-	-	7	0	0	1	2	3	0	4	2	2	2	3	0	0	0	1	2

- prima linie conține indicatorul ce marchează apariția unui page fault sau nu
- a doua linie este reference string-ul
- pentru 20 de accese apar 15 page fault-uri
- FIFO nu previne evacuarea unei pagini ce conține o variabilă utilizată intens, dar care a fost inițializată la începutul programului

Anomalia lui Belady

- considerăm reference string-ul: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- pentru un număr din ce în ce mai mare de frame-uri, ne așteptăm ca numărul de page fault-uri să scadă, dar nu se întâmplă întotdeauna așa (9 fault-uri pentru 3 frame-uri și 10 pentru 4 frame-uri):



Algoritmul optimal de înlocuire a paginilor

- algoritmul are cea mai mică rată de page fault-uri dintre toți algoritmi și nu suferă de anomalia Belady
- principiul: se înlocuiește pagina care va fi folosită cel mai târziu

p	p	p	p	-	p	-	p	-	-	p	-	-	p	-	-	-	p	-
7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0
7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7
-	0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0
-	-	1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1

- apar 9 page fault-uri
- algoritmul nu se poate aplica în practică deoarece presupune cunoașterea în avans a paginilor accesate de program

Algoritmul least-recently used (LRU) de înlocuire a paginilor

- este o aproximare a algoritmului optimal
- principiul: se înlocuiește pagina care a fost folosită cel mai timpuriu (cea care nu a mai fost folosită de cel mai mult timp)
- vom folosi din nou o structură similară cu FIFO, în care elementul găsit este scos și reintrodus la coadă (comportament de tip stivă)

p	p	p	p	-	p	-	p	p	p	p	-	-	p	-	p	-	p	-
7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0
7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0
-	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7
-	-	7	0	1	2	2	3	0	4	2	2	0	3	3	1	2	0	1

- apar 12 page fault-uri
- algoritmul LRU este folosit de regulă în practică și este considerat bun
- nu suferă de anomalia Belady

Aproximări ale algoritmului LRU

- reordonarea paginilor trebuie realizată în HW, altfel penalizarea de performanță este prea mare
- există sisteme care nu dispun de astfel de HW dedicat, suntem forțați să folosim FIFO
- multe sisteme furnizează un **reference bit**, setat în HW de câte ori pagina este referită
- inițial toți biții sunt 0, dar după un timp putem discerne între paginile referite și cele nereferite, fără a cunoaște însă ordinea

Algoritmul additional-reference-bits

- asociază un byte (8 biți) pentru fiecare pagină din tabela de pagini
- la intervale regulate SO shiftează la dreapta fiecare byte asociat paginii și transferă bitul referință pe poziția cea mai semnificativă (MSB)
- o pagină cu byte-ul 1100.0100 a fost utilizată mai recent decât pagina cu byte-ul asociat 0111.0111
- nu este garantat că numerele sunt unice (putem folosi FIFO pentru a le departaja)
- dacă numărul de biți din istoric nu este 8 ci zero, algoritmul devine algoritmul **second-chance**

Algoritmul second-chance

- folosește algoritmul de bază FIFO, luând în considerare și reference bit
- pentru pagina selectată prin algoritmul FIFO, se inspectează bitul reference
- dacă bitul este zero, pagina este aleasă pentru înlocuire
- dacă bitul este unu, paginii "i se mai acordă o șansă", adică bitul este resetat și se trece la următoarea pagină; dacă toată structura FIFO a fost parcursă în acest fel și nu s-a selectat o pagină înainte de a ajunge la sfârșit, se reia de la început
- o variantă de implementare este cea folosind o coadă circulară

Algoritmi bazați pe numărare

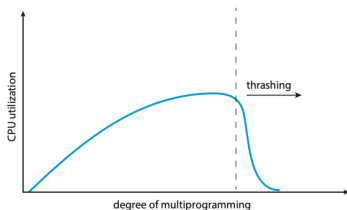
- se folosesc de numărarea referințelor care au avut loc pentru o pagină
- algoritmul Least Frequently Used (LFU) propune ca pagina cu cel mai mic counter să fie înlocuită; motivul este că o pagină des folosită va avea o valoare mare asociată cu acest counter
- pentru LFU, există pericolul unei intense utilizări inițiale urmată de o perioadă de sub-utilizare; se poate utiliza un algoritm de degradare exponențială, prin shiftarea la dreapta la intervale regulate
- algoritmul Most Frequently Used (MFU) se bazează pe raționamentul că pagina cu cel mai mic contor probabil de-abia a fost adusă și urmează să fie utilizată
- utilizările LFU sau MFU nu sunt folosite aproape deloc; calcularea lor este greoaie și nu aproximează bine algoritmul optimal

- 1 Fundamente
- 2 Paginarea la cerere
- 3 Înlocuirea paginilor
- 4 Trashing**
- 5 Alocarea memoriei de către kernel

Cauze

- este posibil ca un proces să nu aibă suficiente frame-uri disponibile, va genera page fault
- se selectează o pagină ce se va înlocui, dar ea este folosită des; ca atare, lipsa ei va genera curând un alt page-fault
- procesul va genera page fault-uri în mod repetat, pentru că evacuează pagini de care are nevoie curând; fenomenul se numește **trashing**
- un proces trashing petrece mai mult timp realizând înlocuirea paginilor decât executându-se
- algoritmi **globali** de înlocuire a paginilor recuperează frame-uri în dauna tuturor proceselor care rulează în sistem
- ex. un sistem care funcționează cu un astfel de algoritm global: numărul mic de procese din coada ready determină sistemul să pună în coadă mai multe procese; mai multe procese înseamnă o concurență mai mare, adică page fault-uri mai dese; unele procese pot ajunge în situația de trashing, numărul de procese din coada ready scade → planificatorul introduce și mai multe procese în coada ready

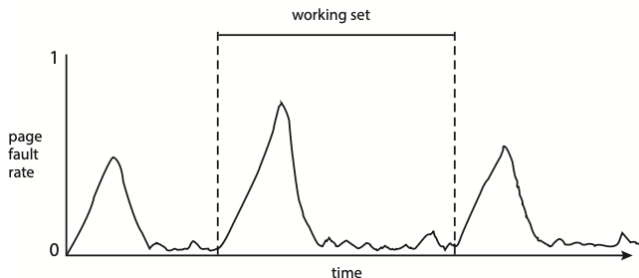
Scăderea gradului de multiprogramare după instalarea trashing-ului



- o creștere a gradului de multiprogramare va accentua trashing-ul, SO va petrece majoritatea timpului în swapping
- se poate evita prin folosirea de algoritmi **locali** de paginare
- dacă un proces intră în trashing, el va îngreuna accesul la memoria secundară și pentru alte procese, deoarece dispozitivul va deveni mai ocupat, iar apelurile I/O vor fi realizate mai lent
- pentru a preveni trashing-ul, unui proces i se vor furniza frame-urile de care are nevoie, i.e. să i se garanteze numărul minim de frame-uri pentru care rata de page fault-uri este acceptabilă
- principiul **localizării** acceselor la memorie afirmă că, pe măsură ce se execută, un proces trece de la o "vecinătate" la alta, adică majoritatea acceselor la memorie se fac în preajma celui anterior

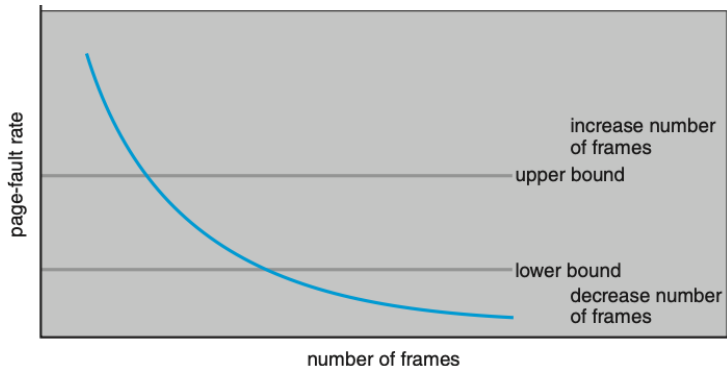
Modelul "working set"

- modelul "working set" se bazează pe presupunerea localizării referințelor
- ideea este să se examineze doar cele mai recente pagini
- pe măsură ce procesul trece de la o localizare la alta, rata de page fault-uri variază



- un vârf de page fault-uri reprezintă trecerea la o nouă localizare; "perioada" dintre două vârfuri marchează tranziția de la o localizare la alta

Frecvența page fault-urilor



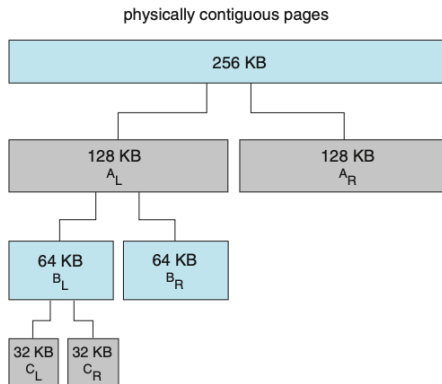
- ideea este să prevenim apariția trashing-ului
- stabilim două limite: când rata page fault-urilor depășește limita superioară mai alocăm frame-uri în plus, iar când este sub cea inferioară, reducem numărul lor
- dacă rata page fault-urilor crește și nu sunt frame-uri disponibile, procesul este swapped-out iar frame-urile eliberate, redistribuite

- 1 Fundamente
- 2 Paginarea la cerere
- 3 Înlocuirea paginilor
- 4 Trashing
- 5 Alocarea memoriei de către kernel**

Buddy systems

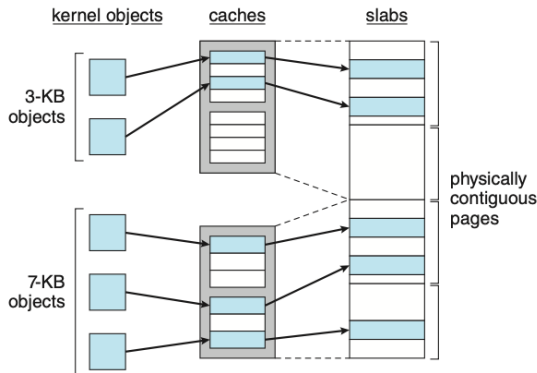
- memoria utilizată de kernel este alocată dintr-un pool diferit de cel folosit pentru alocare frame-urilor pentru paginile proceselor
- motive:
 - ❶ structurile de date folosite de kernel pot fi mult mai mici decât o pagină, s-ar ajunge la fragmentare internă; multe kernel-uri nu folosesc paginarea, performanța ar avea grav de suferit
 - ❷ hardware-ul I/O interacționează cu memoria fizică, fără controlul CPU și al memoriei virtuale, trebuie să se facă în zone de memorie contigue (Direct Memory Access - DMA)
- memoria alocată de **buddy systems** se face dintr-o zonă contiguă (adrese consecutive), blocurile alocate au dimensiuni puteri ale lui 2 (4 KB, 8 KB, 16 KB etc.)
- ex. avem de satisfăcut o cerere de 21 KB; sistemul va împărți zona de 256 KB succesiv în două până obținem cel mai mic bloc care poate cuprinde dimensiunea solicitată

Alocarea folosind buddy systems



- avantajul este că blocurile adiacente de dimensiuni egale pot fi comasate
- fragmentarea internă este mare; fragmentarea internă poate ajunge la 50%

Slab allocation (alocarea pe bucăți mari)



- există un singur cache pentru fiecare structură a kernel-ului (unul pentru descriptorii de proces, altul pentru semafoare, altul pentru descriptorii de fișiere etc.)
- fiecare cache este populat de obiecte care sunt instanțieri ale structurilor kernel-ului

Slab allocation (2)

- numărul de obiecte din cache depinde de dimensiunea unui slab
- un slab de 12 KB (alcătuit din 3 pagini de 4 KB) poate stoca 6 obiecte de 2 KB
- inițial toate obiectele din cache sunt marcate la libere
- obiectele din cache sunt alocate pe măsură ce este necesar
- un slab poate fi alocat **full**, când toate obiectele din el sunt folosite, **empty** sau **partial**
- o cerere este verificată să fie satisfăcută cu ajutorul unui slab partial; dacă nu există, se încearcă unul gol
- beneficii:
 - 1 nu se risipește memorie prin fragmentare; fiecare slab este împărțit în bucăți de mărimea obiectelor folosite de kernel
 - 2 cererile de alocare se satisfac rapid; schema este potrivită pentru cereri de alocare/dealocare frecvente, cum este cazul funcțiilor kernel; obiectele sunt alocate din cache, pre-alocarea este deja făcută

